

Rollr – Phase 1 & 2 Summary

What was built

Phase 1 – Foundation & Auth

- Monorepo: `backend/` (Node + Express, ESM) and `mobile/` (Expo / React Native + TypeScript).
- Auth, 3 roles, all verified against your live Supabase project:
 - Customer – self-registration + login (email/password via Supabase Auth).
 - Mechanic – login only, no signup route. Accounts are issued by admin/seed.
 - Admin – separate login with hardcoded creds (from `.env`) → backend issues a signed JWT.
- Role routing: after login each role lands on its own home screen (customer gets the real Phase 2 flow; mechanic & admin get placeholder home screens with role name + logout).
- Vehicle types in config (not hardcoded in schema): Bicycle, E-Bike, Electric Scooter, Other. Add new types by editing one file – no migration needed.
- Secrets in `.env` (gitignored) with a documented `.env.example` for both apps.
- Seed script creates 1 admin + 1 mechanic + 1 customer and prints credentials.

Phase 2 – Customer Repair Request Flow

- Request Repair screen: vehicle-type selector, problem description, optional photo (Expo Image Picker → uploaded to Supabase Storage `job-photos` bucket), "use my location" button (Expo Location), submit → saved as `Pending`.
- Job Status screen: request details + a visual tracker (`Pending` → `Accepted` → `In Progress` → `Completed`) + cancel button (only while `Pending`).
- Customer home: shows active job status if one's open, otherwise a "Request Repair" button.
- One-active-job rule enforced (second open request returns 409).
- All job data persists in your Supabase Postgres.

Explicitly NOT built (future phases)

Stripe/payments, mechanic job flow, chat, notifications, ratings, admin dashboard. Hook-in points are marked with `TODO (Phase 3...)` comments.

Folder structure

`~/rollr`

```
├─ README.md, SETUP.md, .gitignore
├─ backend/                                # Node + Express API
│   ├── .env / .env.example
│   └─ src/
│       ├── server.js                    # entrypoint
│       ├── app.js                      # express app, middleware, route mounting
│       └─ config/
```

```

|   |   |─ index.js           # loads/validates env
|   |   |─ supabase.js        # service-role admin client + token verify
|   |   |─ vehicleTypes.js    # ← source of truth for vehicle types
|   |─ db/
|   |   |─ schema.sql         # full schema (idempotent)
|   |   |─ migrate.js         # applies schema.sql
|   |   |─ seed.js            # creates test users, syncs vehicle types
|   |   |─ pool.js            # pg pool (SSL via Supabase pooler)
|   |─ middleware/
|   |   |─ auth.js            # authenticate + authorize(role)
|   |   |─ errorHandler.js
|   |─ routes/                # index, auth.routes, jobs.routes
|   |─ controllers/           # auth.controller, jobs.controller
|   |─ services/              # auth.service, jobs.service
└─ mobile/                    # Expo / React Native
    |─ .env / .env.example
    |─ App.tsx, index.ts, app.json, tsconfig.json
    └─ src/
        |─ config/            # env.ts, vehicleTypes.ts (mirror of
backend)
        |─ services/          # supabase.ts, api.ts, jobs.ts
        |─ hooks/useAuth.tsx  # auth context (session + role)
        |─ navigation/        # RootNavigator.tsx, types.ts
        |─ components/        # Button, TextField, ScreenContainer,
StatusBadge, StatusTracker
        |─ theme/
        └─ screens/
            |─ auth/           # Welcome, CustomerLogin, CustomerRegister,
MechanicLogin, AdminLogin
            |─ customer/       # CustomerHome, RequestRepair, JobStatus
            |─ mechanic/       # MechanicHome (placeholder)
            └─ admin/          # AdminHome (placeholder)

```

Database schema

7 tables (Postgres 17.6 on your Supabase project):

Table	Purpose	Status
roles	customer / mechanic / admin	seeded
users	mirrors Supabase auth id; role_id FK; email, full_name, is_active	active
vehicle_types	mirror of config (key, label, sort_order); jobs reference key as text	synced
jobs	customer_id, mechanic_id (nullable – set in Phase 3), vehicle_type_key, description, photo_url, location_lat/lng, status enum, timestamps	active
subscriptions	user_id, plan, status – schema only	future (Stripe)
messages	job_id, sender_id, body – schema only	future (chat)
ratings	job_id, customer_id, mechanic_id, score 1–5 – schema only	future

- job_status enum: pending → accepted → in_progress → completed, plus cancelled.
- updated_at auto-touch triggers on users, jobs, subscriptions.
- Schema is idempotent – safe to re-run `npm run db:migrate`.

API endpoints (live today)

POST /api/auth/register	Customer self-registration
POST /api/auth/admin/login	Admin (hardcoded creds) → JWT
GET /api/auth/me	Profile + role (all 3 roles)
# all below require authenticate + authorize('customer'):	
POST /api/jobs	Create repair request (→ pending)
GET /api/jobs/active	Current active job, or null
GET /api/jobs	My job history
GET /api/jobs/:id	One job (must be mine)
POST /api/jobs/:id/cancel	Cancel (pending only)

Important things to know before Phase 3

1. Auth split: customers/mechanics authenticate client-side via Supabase; the app then calls `GET /api/auth/me` to resolve role. Admin is the exception — backend-issued JWT. The `authenticate` middleware handles both token types.
2. Mechanic flow is the Phase 3 gap. A submitted job sits in the DB as `pending` with `mechanic_id = NULL`. There's currently no mechanic endpoint or UI to list/accept/update jobs — `MechanicHomeScreen` is a placeholder. The marked hook-in: [routes/jobs.routes.js](#) line 16 (`TODO(Phase 3 - Mechanic flow)`) — add [routes/mechanic.jobs.routes.js](#) under `authorize('mechanic')` for accept + status transitions.
3. DB connection quirk: your Supabase direct host is IPv6-only and this Mac has no IPv6, so `DATABASE_URL` uses the IPv4 Session pooler (`aws-1-us-east-1.pooler.supabase.com`). Keep that — don't switch back to the `db.<ref>.supabase.co` direct string here.
4. Vehicle types live in two mirrored files ([backend/src/config/vehicleTypes.js](#) + [mobile/src/config/vehicleTypes.ts](#)). Edit both when adding a type, then re-run `npm run db:seed` to sync the table. Jobs store the key as text, so no migration is ever needed.
5. Storage: public `job-photos` bucket exists with RLS policies (authenticated upload, public read).
6. Real-time isn't wired yet. When you build the mechanic flow, Supabase Realtime on the `jobs` table is the natural way to push new requests to mechanics live — that's the intended Phase 3 mechanism.
7. Secrets: your keys/DB password are in the two `.env` files (gitignored). Fine for dev; rotate before anything public.

Want me to draft the Phase 3 plan (mechanic job queue, accept, status transitions, real-time) when you're ready?